Experiment No. 1
Implement a simple grid-world environment and training an agent using basic Q-learning.
Date of Performance:
Date of Submission:
Marks:
Sign:

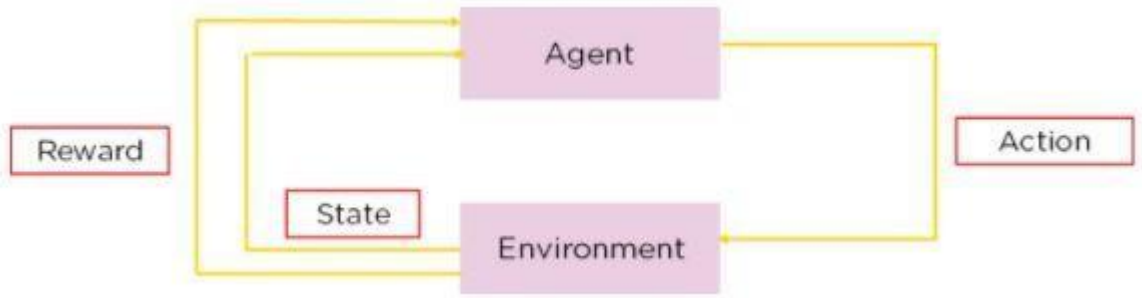
Aim: Implement a simple grid-world environment and training an agent using basic Q-learning.

Objective: To implement the Q-learning update rule and maintain a **Q-table** that stores the expected rewards for each state-action pair.

Theory:

Q-Learning

Q-Learning is a Reinforcement learning policy that will find the next best action, given a current state. It chooses this action at random and aims to maximize the reward.



Q-learning is a model-free, off-policy reinforcement learning that will find the best course of action, given the current state of the agent. Depending on where the agent is in the environment, it will decide the next action to be taken.

The objective of the model is to find the best course of action given its current state. To do this, it may come up with rules of its own or it may operate outside the policy given to it to follow. This means that there is no actual need for a policy, hence we call it off-policy.

Model-free means that the agent uses predictions of the environment's expected response to move forward. It does not use the reward system to learn, but rather, trial and error.

An example of Q-learning is an Advertisement recommendation system. In a normal ad recommendation system, the ads you get are based on your previous purchases or websites you may have visited. If you've bought a TV, you will get recommended TVs of different brands.

Using Q-learning, we can optimize the ad recommendation system to recommend products that are frequently bought together. The reward will be if the user clicks on the suggested product.

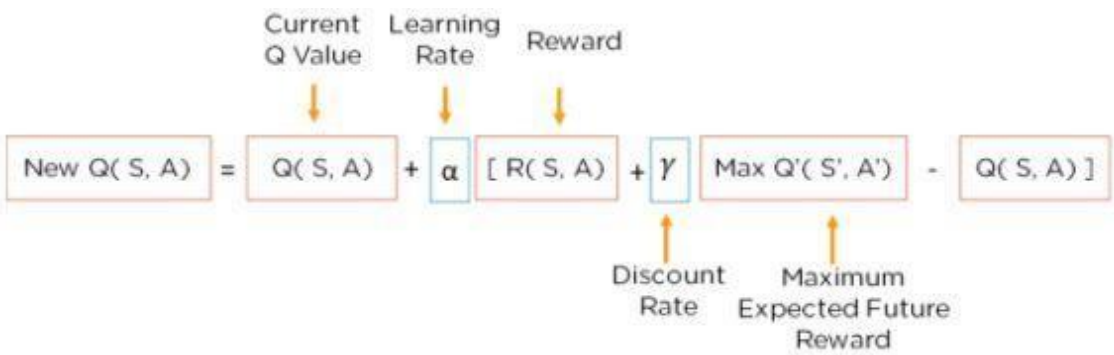
Important Terms in Q-Learning

1. States: The State, S, represents the current position of an agent in an environment.
2. Action: The Action, A, is the step taken by the agent when it is in a particular state.
3. Rewards: For every action, the agent will get a positive or negative reward.
4. Episodes: When an agent ends up in a terminating state and can't take a new action.
5. Q-Values: Used to determine how good an Action, A, taken at a particular state, S, is. Q (A, S).
6. Temporal Difference: A formula used to find the Q-Value by using the value of current state andaction and previous state and action.

What Is The Bellman Equation?

The Bellman Equation is used to determine the value of a particular state and deduce how good it is to be in/take that state. The optimal state will give us the highest optimal value.

The equation is given below. It uses the current state, and the reward associated with that state, along with the maximum expected reward and a discount rate, which determines its importance to the current state, to find the next state of our agent. The learning rate determines how fast or slow, the model will belearning.



How to Make a Q-Table?

While running our algorithm, we will come across various solutions and the agent will take multiple paths. How do we find out the best among them? This is done by tabulating our findings in a table calleda Q-Table.

A Q-Table helps us to find the best action for each state in the environment. We use the Bellman Equation at each state to get the expected future state and reward and save it in a table to compare with other states.

Lets us create a q-table for an agent that has to learn to run, fetch and sit on command. The steps taken to construct a q-table are :

Step 1: Create an initial Q-Table with all values initialized to 0

When we initially start, the values of all states and rewards will be 0. Consider the Q-Table shown below which shows a dog simulator learning to perform actions :

Action	Fetching	Sitting	Running
Start	0	0	0
Idle	0	0	0
Wrong Action	0	0	0
Correct Action	0	0	0
End	0	0	0

Step 2: Choose an action and perform it. Update values in the table

This is the starting point. We have performed no other action as of yet. Let us say that we want the agent to sit initially, which it does. The table will change to:

Action	Fetching	Sitting	Running
Start	0	1	0
Idle	0	0	0
Wrong Action	0	0	0
Correct Action	0	0	0
End	0	0	0

Step 3: Get the value of the reward and calculate the value Q-Value using Bellman Equation

For the action performed, we need to calculate the value of the actual reward and the Q(S, A) value

Action	Fetching	Sitting	Running
Start	0	1	0
Idle	0	0	0
Wrong Action	0	0	0
Correct Action	0	34	0
End	0	0	0

Step 4: Continue the same until the table is filled or an episode ends

The agent continues taking actions and for each action, the reward and Q-value are calculated and itupdates the table.

Implementation:

```
import numpy as np

import random



# Step 1: Define Grid World

grid_size = 4

num_states = grid_size * grid_size

num_actions = 4  # up, down, left, right
```

```
# Actions

actions = ['up', 'down', 'left', 'right']


# Step 2: Initialize Q-table

Q = np.zeros((num_states, num_actions))


# Step 3: Parameters

alpha = 0.1      # learning rate

gamma = 0.9      # discount factor

epsilon = 0.2    # exploration rate

episodes = 500


# Step 4: Reward function

def get_reward(state):

    if state == num_states - 1:  # goal state

        return 10

    return -1  # penalty for each step


# Step 5: State transition

def next_state(state, action):

    row, col = divmod(state, grid_size)
```

```
    if action == 0 and row > 0: row -= 1        # up
    elif action == 1 and row < grid_size - 1: row += 1  # down
    elif action == 2 and col > 0: col -= 1      # left
    elif action == 3 and col < grid_size - 1: col += 1  # right

    return row * grid_size + col

# Step 6: Training
for ep in range(episodes):
    state = 0  # start from top-left

    while state != num_states - 1:
        # Choose action (epsilon-greedy)
        if random.uniform(0,1) < epsilon:
            action = random.randint(0, num_actions-1)
        else:
            action = np.argmax(Q[state])

        new_state = next_state(state, action)
        reward = get_reward(new_state)

        # Q-learning update rule
```

```
        Q[state, action] = Q[state, action] + alpha * (
            reward + gamma * np.max(Q[new_state]) - Q[state, action]
        )

        state = new_state

# Step 7: Display Q-table
print("Q-Table:\n", Q)

# Step 8: Test learned policy
state = 0
path = [state]

while state != num_states - 1:
    action = np.argmax(Q[state])
    state = next_state(state, action)
    path.append(state)

print("Optimal Path:", path)
```

Conclusion:

Q-learning is an effective reinforcement learning algorithm for training agents in unknown environments. It helps the agent learn optimal actions through trial and error using rewards. The grid-world example demonstrates how an agent improves its decision-making over time by updating Q-values.